



GUÍA DE ACTIVIDAD PRÁCTICA

Ingeniería para el Desarrollo Móvil · Unidad 3 · Un plugin como adaptador, de punta a punta

Programa	Facultad de Ingeniería · Posgrado	Asignatura	Ingeniería para el Desarrollo Móvil
Unidad	Unidad 3 · Práctica opcional	Unidad / Corte	3 · Corte 1
Modalidad	Individual o en pareja	Periodo	2026-B
Tipo	Formativa (opcional, sin nota)	Entrega	Aula Moodle, según indique el tutor

Objetivos

- Encapsular un plugin de Capacitor detrás de un puerto del dominio, sin que el negocio importe infraestructura.
- Implementar y verificar en un dispositivo el flujo de permisos just-in-time con sus estados.
- Probar el caso de uso con un fake propio del puerto, de forma portable entre Jasmine/Karma y Vitest.
- Generar un APK de depuración y contrastarlo con el checklist de release del marco de gobierno móvil.

1. Objetivo

Llegar a la **Actividad de aprendizaje 3 (H3, vence el domingo 1 de noviembre, 11:59 p. m.)** con la parte más delicada ya resuelta: **un plugin nativo integrado como adaptador, de punta a punta**. Esta práctica es **opcional y no calificable**. Hazla sobre tu proyecto de aula (uno de tus dos plugins) o, si aún no lo tienes, sobre un proyecto mínimo con Ionic 9 + Angular 22 + Capacitor 8.



Todo es reutilizable. Las partes A y B adelantan el **mínimo 7** del proyecto de aula (dos plugins como adaptadores con permisos *just-in-time*); las partes C y D, el **mínimo 8** (pruebas de casos de uso + binario + checklist de release). Las evidencias van directo a la memoria técnica de la A3.

Elige **un** recurso: cámara o geolocalización (o push o biometría, si tu problema lo exige). Tiempo estimado: de 3 a 4 horas. El ejemplo de referencia es **CapturarEvidencia** de Bitácora de Campo, en la sección 2 de la OVA de la unidad.

2. Parte A — Puerto en el dominio y adaptador Capacitor

```
src/app/features/<tu-feature>/
├─ domain/          puerto (interface) + errores de dominio + caso de uso    ← cero
@capacitor
├─ data/            adaptador que implementa el puerto con el plugin        ← único
import nativo
├─ presentation/    página + ViewModel (depende del dominio, nunca de data/)
<tu-feature>.providers.ts  InjectionToken + useClass del adaptador + useFactory del
caso de uso
```

1. **Puerto:** declara en `domain/` una interfaz en el idioma de tu negocio con `permiso()` y la operación (`tomarFoto()`, `actual()`...). Nada de `PermissionState`, `Photo` ni tipos del plugin en la firma.
2. **Errores de dominio:** `PermisoDenegadoError(recurso, definitivo)` y, si aplica, uno para «recurso no disponible» (sin señal, tiempo agotado).
3. **Adaptador:** en `data/`, la única clase que importa `@capacitor/....`. Traduce `granted`, `limited`, `prompt`, `prompt-with-rationale` y `denied` a tus estados, y los errores del plugin (cancelación, tiempo agotado) a errores de dominio.
4. **Caso de uso:** en `domain/`, con una **regla de negocio explícita** sobre el recurso: ¿es obligatorio o de mejor esfuerzo? Escríbela en una línea antes de programar.
5. **Composición:** `InjectionToken` y proveedores en el archivo de `providers` de la feature, no en el dominio.



Verifica la regla de dependencias con una búsqueda; ambas deben salir **vacías**:

```
grep -rn "@capacitor/" src/app/features/*/domain
grep -rn "/data/" src/app/features/*/presentation
```

Cuidado: Una interfaz que devuelve `Promise<Photo>` o recibe `CameraResultType` no es un puerto: es el plugin con otro nombre. Si cambiar de plugin te obligaría a tocar el dominio, la abstracción tiene fuga.

3. Parte B — Flujo de permisos just-in-time

Requisitos:

- El permiso se pide **solo** cuando el usuario dispara la función, nunca al abrir la app.
- Si el estado es **preguntable**, se muestra **tu** pantalla de justificación antes del diálogo del sistema, con un «Ahora no» que deja la app en un estado degradado y útil.
- Si es **denegado**, se explica qué se pierde y se ofrece ir a los ajustes de la app: **ningún bucle de diálogos**.
- `AndroidManifest.xml` declara solo lo que usas; `Info.plist` lleva un texto de propósito concreto (qué hace la app con el recurso), no «la app necesita la cámara».

Prueba los cuatro escenarios en un dispositivo o emulador Android y guarda una captura de cada uno:

Escenario	Cómo provocarlo	Resultado esperado
Concedido	Primera vez → Continuar → Permitir	La función opera sin interrupciones
Negado una vez	Primera vez → Continuar → No permitir	Al reintentar, justificación más concreta y nuevo diálogo



Negado definitivo	Negar dos veces (Android 11+)	Explicación + acceso a ajustes; la app no se bloquea
Revocado desde Ajustes	Conceder y luego quitarlo en Ajustes > Apps	La app detecta el cambio y vuelve al flujo sin fallar

Consejo. Para repetir los escenarios, desinstala y reinstala la app, o revoca el permiso con `adb shell pm revoke <applicationId> android.permission.CAMERA`. Si tienes un iPhone con Xcode, repite el negado definitivo en iOS: allí basta **una** negativa.

4. Parte C — Prueba del caso de uso con un fake del puerto

Requisitos:

- Un **fake propio** del puerto: una clase que implementa la interfaz, con el resultado configurable y un contador de llamadas. Sin `jasmine.createSpyObj`, `spyOn`, `vi.fn` ni `expectAsync`.
- Al menos **cuatro pruebas** con nombres `unidad_condicion_resultado`: el camino feliz; el permiso negado que produce el error de dominio; la regla de negocio (dato opcional ausente o segundo recurso que no se pide); y un error inesperado que **se propaga** en lugar de tragarse.
- Determinismo: si el caso de uso registra la hora, el tiempo entra por un puerto Reloj falso.
- `npm run test:ci` en verde y el informe de coverage/ con **≥ 80 % en `domain/`**.

```
// Molde portable (corre igual con Jasmine/Karma y con Vitest)
it('<casoDeUso>_<condicion>_<resultado>', async () => {
  // Arrange: fakes del puerto con el resultado que exige el escenario
  // Act: ejecutar el caso de uso; capturar el rechazo con try/catch si se espera un error
  // Assert: UNA afirmación lógica con toBe / toEqual / toBeNull
});
```

Extensión recomendada: una prueba del ViewModel con `TestBed`, reemplazando el token del puerto y el caso de uso por fakes, que verifique la transición a la pantalla de justificación.



5. Parte D — APK de depuración y checklist de release

```
npm run build && npx cap sync android
cd android && ./gradlew assembleDebug
adb install -r app/build/outputs/apk/debug/app-debug.apk
adb shell am start -W -n <applicationId>/.MainActivity # anota TotalTime (arranque)
```

Ese APK lo firma automáticamente la **clave de depuración** del SDK: sirve para instalar y probar, **no** para publicar. Contrástalo con el checklist de release del marco y marca lo que ya cumples:

- ☐ Pruebas unitarias en verde con `npm run test:ci`; cobertura de `domain/` anotada.
- ☐ Solo los permisos usados en el manifest y en el plist.
- ☐ `capacitor.config.ts` sin `server.url` ni `cleartext: true`.
- ☐ Ningún token, llave ni contraseña en el repositorio ni en `environment.ts`.
- ☐ `versionName` (SemVer) y `versionCode` definidos en `build.gradle`.
- ☐ Tamaño del APK y tiempo de arranque anotados como **línea base** de tus presupuestos.

Y escribes, en tres líneas, **lo que falta para H3**: keystore de release fuera del repositorio, AAB/APK firmado en modo release, IPA documentado o simulado, pipeline (real o diseñado) y cómo observarás la app publicada.

6. Entrega y autoevaluación

- Documento **PDF de 2 a 3 páginas** (o un `docs/practica-u3.md` en tu repositorio con el enlace): diagrama del puerto y el adaptador, la regla de negocio en una línea, las cuatro capturas de la Parte B, la salida de `npm run test:ci` con la cobertura de `domain/`, el tamaño del APK, el tiempo de arranque y el checklist marcado.



- Nombre sugerido: U3-practica-<apellido>.pdf; en pareja, ambos nombres en la primera línea.
- Se sube donde indique el tutor. Al ser práctica opcional **no tiene nota**: su valor es que la memoria técnica de la A3 se escriba sobre evidencia que ya existe.

Antes de entregar:

- [] Ningún archivo de domain/ ni de presentation/ importa @capacitor/ ni data/.
- [] El puerto habla el idioma del negocio y el adaptador traduce estados y errores.
- [] Los cuatro escenarios de permisos están probados en dispositivo, con captura.
- [] Las pruebas usan fakes propios, tienen nombres unidad_condicion_resultado y pasan con test:ci.
- [] El APK se instaló y quedaron anotadas las líneas base de tamaño y arranque.

Criterio	Logrado	En proceso	Insuficiente
Puerto y adaptador	Dominio sin imports nativos; el adaptador traduce estados y errores	Puerto correcto, pero con tipos del plugin en la firma	El componente llama al plugin directamente
Permisos just-in-time	Cuatro escenarios probados; justificación y ruta a ajustes	Pide en el momento correcto, sin justificación propia	Pide al arrancar o entra en bucle
Pruebas	Fake propio, cuatro pruebas portables, ≥ 80 % en domain/	Pruebas con utilidades del runner o cobertura bajo el piso	Sin pruebas del caso de uso
Binario y checklist	APK instalado, checklist marcado y brecha a H3 escrita	APK instalado sin checklist	Sin binario

Nota: Versión imprimible. [Descarga esta práctica en PDF](pdf/Practica-Unidad03.pdf), con el membrete institucional.